



Pods: Init Containers & Multi-Container Patterns

Composing Pods from Cooperating Containers

Intermediate Kubernetes — Pods Deep Dive (companion to Module 1)

Objectives

- Explain what containers in a Pod actually share — network, storage, lifecycle
- Use **init containers** for ordered, run-to-completion setup
- Build **multi-container Pods** with the sidecar pattern
- Recognize the ambassador and adapter patterns
- Use **native sidecars** (restartPolicy: Always init containers, 1.29+)
- Target a specific container with `kubectl logs -c / exec -c`

A Pod Is a Team of Containers

The smallest deployable unit — not "one container"



What Containers in a Pod Share

Shared

- **Network namespace** — same IP, reach each other on `localhost`
- **Volumes** — mount the same `emptyDir/PVC`
- **Lifecycle & node** — scheduled together, live and die together

Not shared (by default)

- **Filesystem** — each container has its own image rootfs
- **Process namespace** — isolated unless `shareProcessNamespace`
- **Restarts** — one container can restart without the others

Because they share the network and volumes, containers in a Pod can cooperate closely — that is the whole point of putting them together.



Init Containers

Ordered, run-to-completion setup before the app starts



Init Containers — How They Work

Rules

- Run **in order**, one at a time
- Each must **succeed** before the next starts
- App containers wait until **all** are done
- A failure is retried (per `restartPolicy`)

Good for

- Waiting for a dependency (DB, service) to be ready
- Fetching config, secrets, or templates
- Running schema migrations
- Setting permissions / seeding a shared volume

Declared under `spec.initContainers`. They share the Pod's volumes, so they can prepare state the app container then consumes.

Multi-Container Patterns

Helpers that run alongside the main container

The Sidecar Pattern

A helper container that extends the main one

Runs **concurrently** in the same Pod, cooperating through the shared network and volumes — the main container stays focused on its job.

- **Log shipper** — tails the app's log file and forwards it
- **Content sync** — pulls/writes content the app serves
- **Proxy / service mesh** — Envoy intercepting traffic (e.g. Istio)

In the lab, a content-sync sidecar rewrites the page every 10s into a shared volume that nginx serves.



Ambassador & Adapter

Ambassador

Proxies the app's **outbound** connections. The app talks to `localhost`; the ambassador handles sharding, retries, or TLS to the real backend.

e.g. a local Redis/DB proxy

Adapter

Normalizes the app's **output** for the outside world — reformatting logs or exposing metrics in a standard shape.

e.g. a Prometheus exporter sidecar

All three (sidecar, ambassador, adapter) are just multi-container Pods — they differ in *which direction* the helper faces.

✨ Native Sidecars (1.29+)

A **native sidecar** is an `initContainers` entry with `restartPolicy: Always`. It starts **before** the app, runs **for the app's lifetime**, and does **not** block Pod completion.

Fixes plain-sidecar gaps

- Guaranteed running before the app starts
- A Job's Pod can now actually *complete*
- Shuts down after the app on termination

Ideal for

- Log shippers & metrics agents
- Service-mesh proxies
- Config / secret refreshers

Which One Do I Reach For?

Need	Use
One-time setup / wait for a dependency, then exit	Init container
Helper that runs for the app's whole life	Native sidecar (initContainers + restartPolicy: Always)
Two peers cooperating, ordering not critical	Plain multi-container Pod
Truly independent workloads	Separate Pods (don't over-pack)

Only co-locate containers that must share a lifecycle or a volume. When in doubt, separate Pods.



Lab: Init Containers & Multi-Container Pods

Hands-on: compose Pods from cooperating containers

- Use an **init container** to prepare a shared volume before nginx starts
- Build a **sidecar** Pod (2/2) that refreshes the page nginx serves
- Run a **native sidecar** log shipper (`restartPolicy: Always`)
- Target individual containers with `kubectl logs -c / exec -c`

Duration: ~25-35 minutes

Key Takeaways

- A **Pod** is a group of co-scheduled containers sharing a network namespace (`localhost`) and volumes.
- **Init containers** run to completion, in order, *before* app containers — setup and dependency gating.
- **Sidecars** run alongside the app to extend it; **ambassador/adapter** are direction-specific variants.
- **Native sidecars** (`initContainers + restartPolicy: Always`) start first, live with the app, and don't block completion.
- `kubectl logs -c` and `exec -c` target one container in a multi-container Pod.